Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «Модели решения задач в И С»
Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнил:
Студент 3 курса
Группы ИИ-26
Шоева Е.Т.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провести сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Ход работы

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:
   - Заменить пороговую функцию активации на сигмоидную.
   - Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
   - Реализовать вычисление градиента ŷ-exi для обновления весов.
2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

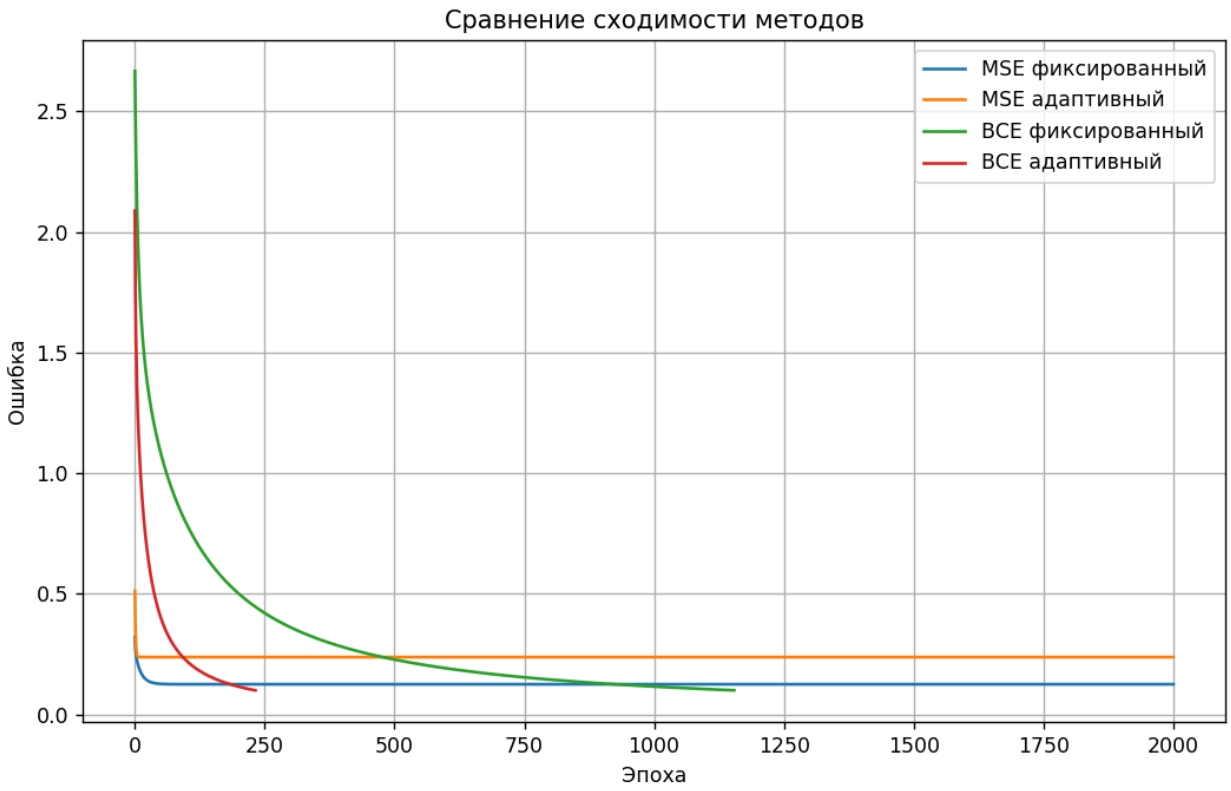
3. Реализовать обучение с **адаптивным** шагом:
   - На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
   - Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
   - Сохранять значения Esp для построения графика.
4. Провести сравнительный анализ:
   - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
   - MSE + фиксированный шаг (данные из ЛР №1);
   - MSE + адаптивный шаг (данные из ЛР №2);
   - BCE + фиксированный шаг;
   - BCE + адаптивный шаг.
   - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).
5. Визуализировать результаты:
   - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
   - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
   - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.
6. Реализовать режим функционирования:
   - Пользователь вводит координаты x1,x2.
   - Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
   - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).
7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.
8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

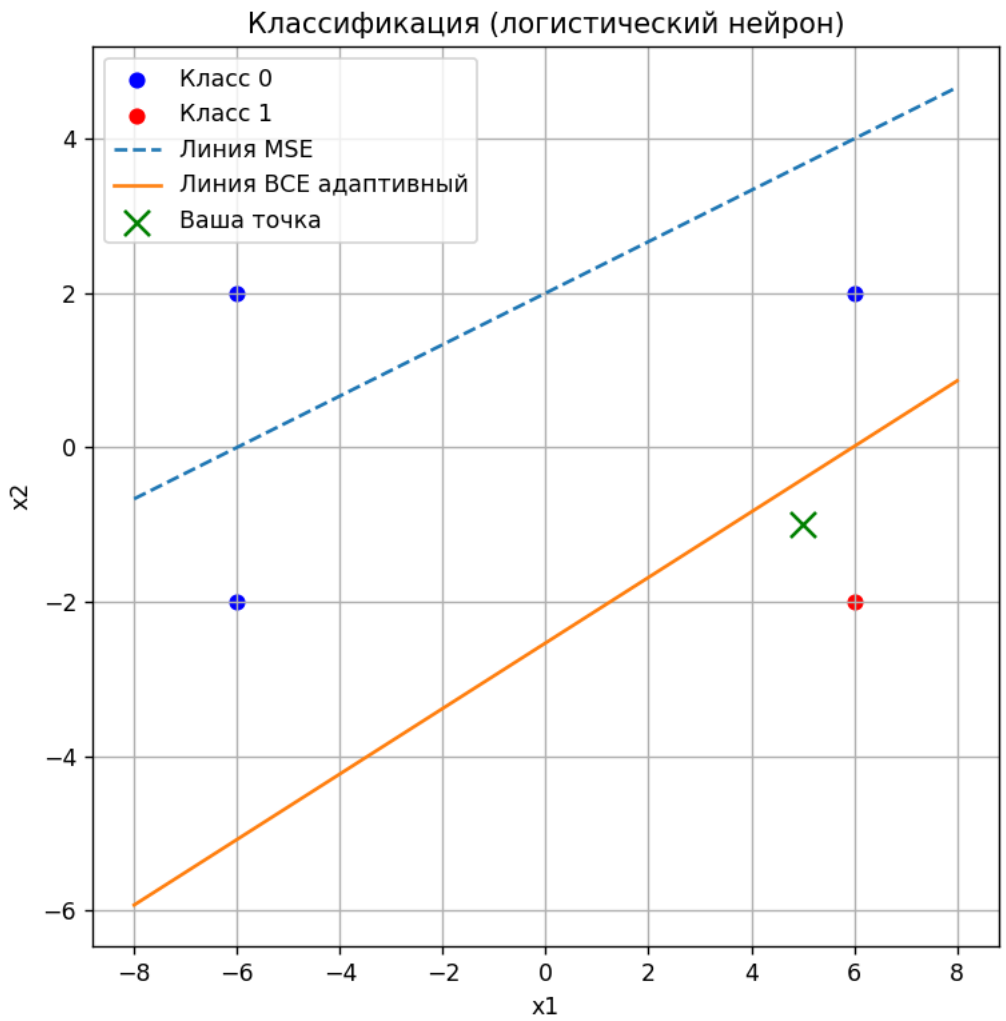
```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

X = np.array([
    [6, 2],
    [-6, 2],
    [6, -2],
    [-6, -2]
], dtype=float)

y = np.array([0, 0, 1, 0], dtype=float)
```

```
X = X / 6.0
X_bias = np.hstack([X, np.ones((X.shape[0], 1))])

Ee_mse = 0.01
Ee_bce = 0.1

max_epochs = 2000
lr = 0.1

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def mse(y_true, y_pred):
    return 0.5 * np.sum((y_true - y_pred) ** 2)

def bce(y_true, y_pred):
    eps = 1e-8
    return -np.sum(y_true*np.log(y_pred+eps) +
                   (1-y_true)*np.log(1-y_pred+eps))

w_mse_fixed = np.random.randn(3) * 0.1
Es_mse_fixed = []
epochs_mse_fixed = 0

while True:
    y_pred = X_bias @ w_mse_fixed
    error = y - y_pred
    Es = mse(y, y_pred)
    Es_mse_fixed.append(Es)

    if Es <= Ee_mse or epochs_mse_fixed >= max_epochs:
        break

    w_mse_fixed += lr * (X_bias.T @ error)
    epochs_mse_fixed += 1

print("Эпохи MSE фиксированный =", epochs_mse_fixed)

w_mse_adapt = np.random.randn(3) * 0.1
Es_mse_adapt = []
epochs_mse_adapt = 0

while True:
    for i in range(len(X_bias)):
        xi = X_bias[i]
        yi = y[i]

        y_pred = np.dot(xi, w_mse_adapt)
        error = yi - y_pred

        alpha = 1 / (1 + np.sum(xi[:-1]**2))
        w_mse_adapt += alpha * error * xi

    y_all = X_bias @ w_mse_adapt
    Es = mse(y, y_all)
    Es_mse_adapt.append(Es)

    if Es <= Ee_mse or epochs_mse_adapt >= max_epochs:
        break

    epochs_mse_adapt += 1

print("Эпохи MSE адаптивный =", epochs_mse_adapt)
```

```
w_bce_fixed = np.random.randn(3) * 0.1
Es_bce_fixed = []
epochs_bce_fixed = 0

while True:
    y_pred = sigmoid(X_bias @ w_bce_fixed)
    error = y_pred - y
    Es = bce(y, y_pred)
    Es_bce_fixed.append(Es)

    if Es <= Ee_bce or epochs_bce_fixed >= max_epochs:
        break

    w_bce_fixed -= lr * (X_bias.T @ error)
    epochs_bce_fixed += 1

print("Эпохи BCE фиксированный =", epochs_bce_fixed)

w_bce_adapt = np.random.randn(3) * 0.1
Es_bce_adapt = []
epochs_bce_adapt = 0

while True:
    for i in range(len(X_bias)):
        xi = X_bias[i]
        yi = y[i]

        y_pred = sigmoid(np.dot(xi, w_bce_adapt))
        error = y_pred - yi

        alpha = 1 / (1 + np.sum(xi[:-1]**2))
        w_bce_adapt -= alpha * error * xi

    y_all = sigmoid(X_bias @ w_bce_adapt)
    Es = bce(y, y_all)
    Es_bce_adapt.append(Es)

    if Es <= Ee_bce or epochs_bce_adapt >= max_epochs:
        break

    epochs_bce_adapt += 1

print("Эпохи BCE адаптивный =", epochs_bce_adapt)

plt.figure(figsize=(10,6))
plt.plot(Es_mse_fixed, label="MSE фиксированный")
plt.plot(Es_mse_adapt, label="MSE адаптивный")
plt.plot(Es_bce_fixed, label="BCE фиксированный")
plt.plot(Es_bce_adapt, label="BCE адаптивный")
plt.xlabel("Эпоха")
plt.ylabel("Ошибка")
plt.title("Сравнение сходимости методов")
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(7,7))

X_plot = X * 6

for i in range(len(X_plot)):
    if y[i] == 0:
        plt.scatter(X_plot[i][0], X_plot[i][1],
                    color='blue', label='Класс 0' if i == 0 else "")
```

```
    else:
        plt.scatter(X_plot[i][0], X_plot[i][1],
                    color='red', label='Класс 1')

x_vals = np.linspace(-8, 8, 200)
x_vals_norm = x_vals / 6

y_line1 = -(w_mse_fixed[0]*x_vals_norm + w_mse_fixed[2]) / w_mse_fixed[1]
y_line1 = y_line1 * 6
plt.plot(x_vals, y_line1, '--', label="Линия MSE")

y_line2 = -(w_bce_adapt[0]*x_vals_norm + w_bce_adapt[2]) / w_bce_adapt[1]
y_line2 = y_line2 * 6
plt.plot(x_vals, y_line2, label="Линия BCE адаптивный")

x1 = float(input("Введите x1: "))
x2 = float(input("Введите x2: "))

user = np.array([x1/6, x2/6, 1])
prob = sigmoid(np.dot(user, w_bce_adapt))
cls = 1 if prob >= 0.5 else 0

print("Вероятность =", prob)
print("Класс =", cls)

plt.scatter(x1, x2, color='green', marker='x',
            s=120, label="Ваша точка")

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Классификация (логистический нейрон)")
plt.legend()
plt.grid()
plt.show()
```





Метод обучения	Необходимость подбора α	Устойчивость обучения	Скорость сходимости	Число эпох
MSE + фиксированный	требуется	низкая	очень низкая	2000
MSE + адаптивный	не требуется	средняя	низкая	2000
BCE + фиксированный	требуется	высокая	средняя	1154
BCE + адаптивный	не требуется	очень высокая	высокая	232

Вывод: Изучила принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провела сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).